WEEK 2 · LECTURE 1

How Values Are Actually Stored

The Fundamental Types

TODAY'S AGENDA

Four things by the end of today

1

Variables, literals, and operators

Declare a value, write it in every notation C++ offers, and combine both into expressions.

2

The integer family

char, short, int, long, long long — and their unsigned forms.

3

Floating point, briefly

float vs. double, and why this course defaults to double.

4

Fixed-width types

<cstdint>, for when ‘typical’ isn't good enough.

LAST WEEK, REVISITED

What IS an int, actually?

You wrote int items; and double price; last week — without ever asking what an int actually IS. How many bytes does an int take?

- Confident guesses are about to disagree with each other.
- That's the whole point of today.

The same bits, read two ways

As an int

The bits 0x40490FDB
read as an integer:
1,078,530,011

As a float

The exact same bits,
read as a float:
3.14

> **Key idea** A type is an interpretation rule plus a size. The memory never changes — only the interpretation does.

```
std::cout << sizeof(bool);   // prints 1
```

Declare, initialize, and lock a value

```
int score;                      // declared -- UNINITIALIZED
int score = 0;                  // declared AND initialized
const double PI = 3.14159;      // a constant -- cannot be reassigned
```

Key idea Always initialize. An uninitialized variable holds whatever bits were already there — not zero.

REFERENCES

Another name for the same variable

```
int x = 5;
int& ref = x;    // ref is just another name for x -- not a copy
ref = 10;
std::cout << x;    // 10 -- changing ref changed x
```

Key idea int& binds ref to x for good — never rebound afterward. This & (‘reference to’) is a different job from the address-of & you'll meet in week 7.

LVALUE AND RVALUE

Not every value can sit on the left of =

```
int x = 5;     // x is an lvalue -- it has an address
x = 10;        // fine: assigning INTO an lvalue
// 5 = x;      // ERROR: 5 is an rvalue -- nothing to assign into
```

Key idea If you could take its address (&x), it's an lvalue. A bare literal, or most calculation results, is an rvalue — it exists just long enough to be used.

NUMERIC LITERALS

42, 052, 0x2A — same value, different notation

```
int decimal = 42;
int octal   = 052;    // leading 0 -- octal, also 42
int hex     = 0x2A;   // leading 0x -- hex, also 42
```

Key idea A leading 0 means OCTAL, not decimal — 052 is 42, not fifty-two. It's a real, still-live footgun. Hex, by contrast, is everywhere in real code — colors, permissions, addresses — because exactly 2 hex digits make 1 byte.

LITERAL SUFFIXES

Force a different default type

```
long       a = 100L;    // force long
unsigned   b = 100U;    // force unsigned
long long  c = 100LL;   // force long long
float      d = 3.14f;   // force float -- plain 3.14 would be double
```

Key idea Every floating-point literal is a double BY DEFAULT, full stop — the f suffix is the ONLY way to get a float. That's exactly why this course's convention is double everywhere — it's already what you get for free.

MORE DECIMAL NOTATIONS

Same value, different way to write it

```
double a = 3.14;
double b = .5;        // same as 0.5
double c = 5.;        // same as 5.0
double d = 2.5e3;     // scientific notation -- 2.5 x 10^3 -- 2500
```

Discuss What does 2.5e-3 mean? (2.5 x 10^-3 — 0.0025.)

ARITHMETIC OPERATORS

+ - * / % on two real numbers

```
int a = 7, b = 2;
std::cout << a + b;   // 9
std::cout << a - b;   // 5
std::cout << a * b;   // 14
std::cout << a / b;   // 3   -- more on this next lecture
std::cout << a % b;   // 1   -- modulo: the remainder
```

Key idea % is modulo — the remainder left after division. a / b landing on 3, not 3.5, isn't a mistake — it's next lecture's centerpiece.

ASSIGNMENT IS AN EXPRESSION

That's what makes chained assignment legal

```
int a, b, c;
a = b = c = 5;   // right to left: c=5, then b=(c=5), then a=(b=5)
std::cout << a << b << c;   // 555
```

Key idea = evaluates to the value it just assigned. b = c = 5 is itself an expression that evaluates to 5 — = is right-associative, so it groups right to left.

COMPOUND ASSIGNMENT

Read, operate, write back — in one step

```
int total = 0;
total = total + 5;    // total is now 5
total += 5;           // exactly the same thing -- total is now 10
```

Key idea -=, *=, /=, and %= all work the same way: target OP= value means target = target OP value.

Same change to i, different expression value

```
int i = 5;
std::cout << i++;   // prints 5 -- the OLD value, THEN i becomes 6

int j = 5;
std::cout << ++j;   // j becomes 6, THEN prints 6 -- the NEW value
```

Key idea Postfix hands back a COPY of the value before incrementing. Prefix increments first, then hands back the variable itself, already updated.

THE COMMA OPERATOR

Evaluate the left, discard it, return the right

```
int a = (1, 2, 3);          // evaluates 1, then 2, then 3 -- a is 3
int x = 0;
int y = (++x, x + 10);      // x becomes 1, then y = 1 + 10 -- y is 11
```

Key idea The comma in int x = 0, y = 1; is NOT this operator — that's just declaration-list syntax. You'll rarely write the real one on purpose outside a for loop's update clause (week 3) — but now you'll recognize it.

WHAT IS AN EXPRESSION?

Any combination that evaluates to one value

```
7                    // an expression
score                // an expression
score + 7            // an expression
score = 7            // also an expression -- evaluates to 7
```

Key idea An expression is any combination of literals, variables, and operators the compiler can evaluate down to a single value.

Which operator runs first, and which way do ties break

```
std::cout << 2 + 3 * 4;      // 14 -- * binds tighter than +
std::cout << 20 - 5 - 3;     // 12 -- left to right
int a = b = 5;               // right to left
```

Key idea So far, high to low: * / %, then + -, then = += -= etc. (right-to-left), then the comma operator. When in doubt, parenthesize.

SEQUENCE POINTS

A guardrail, not a trick

```
int i = 0;
i = i++ + 1;    // UNDEFINED BEHAVIOR -- never write this
```

Key idea Never modify the same variable more than once in an expression without a sequence point (a semicolon, &&/||, a function call) between the modifications.

WORKED EXAMPLE

Step through precedence out loud

```
int result = 2 + 3 * 4 - 1;
// * first:                2 +  12  - 1
// then left-to-right +/-:     14  - 1
// result:                       13
```

Discuss What does int x = 10 % 3 + 2 * 4; evaluate to? (10 % 3 is 1, 2 * 4 is 8, then 1 + 8 — 9.)

<< and >> chain, just like + does

```
int age;
std::cout << "Enter your age: ";
std::cin >> age;
std::cout << "You are " << age << " years old.\n";
```

Discuss std::cout << a << b; with a = 3 and b = 4 — what prints?

PRINTING EVERY SIZE AT ONCE

In a scratch file, all five in one shot

```
std::cout << "char: "      << sizeof(char)      << "\n";
std::cout << "short: "     << sizeof(short)     << "\n";
std::cout << "int: "       << sizeof(int)       << "\n";
std::cout << "long: "      << sizeof(long)      << "\n";
std::cout << "long long: " << sizeof(long long) << "\n";
```

Key idea On this course's WSL2/Debian x86-64 machines you'll see 1, 2, 4, 8, 8. Say it every time: ‘typical here, not guaranteed everywhere.’

THE INTEGER FAMILY

Guaranteed order, not guaranteed size

1. **char**
   1 byte typical — the smallest integer type.
2. **short**
   2 bytes typical — rarely used by name in this course.
3. **int**
   4 bytes typical — the default; use this unless you have a reason not to.
4. **long**
   8 bytes typical — for when int's range genuinely isn't enough.
5. **long long**
   8 bytes typical, ≥64 bits guaranteed — the safest ‘definitely big enough.’

WORDS WORTH KNOWING

Guaranteed vs. typical

The standard guarantees only a MINIMUM size and an ORDER — never an exact size.

What you're about to see on screen is a fact about THIS machine, not a fact about C++ itself. On this course's WSL2/Debian x86-64 machines: char, short, int, long, long long are 1, 2, 4, 8, 8 bytes.

> **Discuss** Take this exact source file to an old embedded microcontroller. Could int come out as 2 bytes instead of 4?

SIGNED VS. UNSIGNED

Same bytes, different range

Signed

Can hold negative numbers.
Roughly half the positive
range of unsigned.

Unsigned

Never negative.
Roughly double the positive
range of signed.

```
unsigned int x = 0;
x = x - 1;
std::cout << x;   // NOT -1
```

THE WRAPAROUND

4,294,967,295 — not −1

```
unsigned int x = 0;
x = x - 1;
std::cout << x;    // 4294967295
```

> **Why this matters** That number is not a bug in the compiler — it's unsigned int doing exactly what it's defined to do. Why it happens, and why it's one of the most dangerous bugs you'll write this semester, is next lecture's centerpiece.

THE FLOATING-POINT FAMILY, BRIEFLY

float vs. double

float

Smaller, less precise. Mattered when memory was scarce.

double

This course's default, for essentially everything.

> **From last lecture** 0.1 + 0.2 == 0.3 printed NOT equal. Not a compiler bug — a fact about how every language's floating point works. Full explanation: next lecture.

FIXED-WIDTH TYPES

When 'typical' isn't good enough

```
#include <cstdint>

int32_t  a;   // exactly 32 bits, always
uint8_t  b;   // exactly 8 bits, unsigned, always
uint64_t c;   // exactly 64 bits, unsigned, always
```

`int32_t`

32 bits, signed, always.

`uint8_t`

8 bits, unsigned, always.

`uint64_t`

64 bits, unsigned, always.

RECAP

One number in memory, several interpretations

- A variable must be declared and initialized; a reference is another name for an existing one; only an lvalue can sit on the left of =.
- Numeric literals come in many notations — hex/octal/suffixes; a floating-point literal is double by default unless suffixed f.
- = is itself an expression (chained assignment); ++/-- differ by prefix vs. postfix; precedence and associativity decide evaluation order.
- An expression is any combination of literals, variables, and operators that evaluates to one value.
- A type is a size plus an interpretation rule; int/long/etc. guarantee only a MINIMUM size and an ORDER.
- unsigned buys range by giving up negative numbers — and it wraps, it doesn't error.
- <cstdint> exists for the moment 'typical' isn't a strong enough promise.



Next lecture

sizeof, overflow, and 0.1 + 0.2

The mystery from last week — finally answered.